

Esercitazione 10 - Parte A

15 maggio 2026

Lo scopo di questa esercitazione è quello di implementare una classe `Polynomial` per rappresentare polinomi in una singola variabile. Questa classe deve permettere di svolgere la somma, la sottrazione, il prodotto e l'elevamento a potenza di polinomi, di valutarne il valore in un punto dato, di accedere ai coefficienti usando come indice il grado e di ottenerne la derivata.

Implementazione

```
def __init__(self, coeffs):
```

Un oggetto di tipo `Polynomial` deve poter essere inizializzato usando un dizionario che associa a ogni grado del polinomio il corrispondente coefficiente. Per esempio il dizionario `{0: 1, 2: 6, 4: -2}` indicherà il polinomio $-2x^4 + 6x^2 + 1$.

```
def __getitem__(self, k):
```

```
def __setitem__(self, k, v):
```

Questi due metodi devono permettere l'accesso e la modifica ai coefficienti del polinomio. In questo caso l'indice rappresenta il grado mentre la chiave il coefficiente. Quindi `p[3] = 2.4` indica che il coefficiente di x^3 verrà impostato a 2.4.

```
def __call__(self, x):
```

Questo metodo deve restituire il valore del polinomio valutato nel punto x fornito come argomento.

```
def __add__(self, other):
```

```
def __sub__(self, other):
```

```
def __mul__(self, other):
```

```
def __pow__(self, n):
```

Questi metodi devono permettere la somma, sottrazione e prodotto di polinomi (o di un polinomio con un intero) e l'elevamento a potenza del polinomio (notate che potete sfruttare il fatto di aver definito la moltiplicazione di polinomi per implementare l'elevamento a potenza). Tutti i metodi devono ritornare un nuovo oggetto di tipo `Polynomial` e non modificare `self`.

```
def derivative(self):
```

Questo metodo deve ritornare un oggetto di tipo `Polynomial` rappresentante la derivata dell'oggetto su cui è stato chiamato il metodo (i.e., `self`).

```
def __str__(self):
```

Questo metodo deve ritornare una rappresentazione in stringa del polinomio. Per esempio per il polinomio $2x^4 + 5x^3 + 6x - 2$ una rappresentazione come `2x^4 + 5x^3 + 6x^1 + -2x^0` è sufficiente.

```
def newton_raphson(p, x, n_iter=20):
```

Questa funzione (non metodo) prende come argomenti un polinomio, un punto iniziale e (opzionale, default 20) un numero di iterazioni. La funzione implementa il metodo di Newton-Raphson per trovare gli zeri di una funzione. Si ricorda che il metodo funziona come segue: partendo da una stima iniziale x_0 per uno zero della funzione f si itera nel seguente modo:

$$x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)}$$

Sfruttare alcuni dei metodi implementati nello scrivere questa funzione.

Note

- È possibile (e consigliato) utilizzare la struttura dati `defaultdict` fornita alla libreria standard di Python aggiungendo `from collections import defaultdict`. Il costruttore del `defaultdict` prende come argomento una funzione da chiamare per costruire i valori mancanti, in questo caso è sufficiente utilizzare `defaultdict(int)`, dato che `int()` restituisce 0.

Esercitazione 10 - Parte B

15 maggio 2026

Lo scopo di questa esercitazione è quello di realizzare un sistema per costruire e valutare espressioni. Il sistema dovrà sfruttare il fatto che è possibile definire gerarchie di classi per evitare la duplicazione di codice.

L'idea principale è quella di realizzare un sistema tale per cui una espressione come la seguente:

$2 \times 3 *$

possa venire interpretata come $(2 + x) \times 3$. Per fare questo è necessario utilizzare uno stack (la cui implementazione è fornita nella classe `Stack`) in cui:

- Se si incontra una stringa interpretabile come un intero si crea un oggetto di tipo `Costante` si inserisce nello stack;
- Se si incontra una stringa non corrispondente ad alcuna funzione (e.g., `x` o `y`), si crea un oggetto di tipo `Variable` e si inserisce nello stack;
- Se si incontra una stringa corrispondente a un operatore `op` (e.g., `+`), si verifica l'arità k della funzione (e.g., 2 per `+`), si effettuano k pop dallo stack per ottenere gli argomenti di `op`, si costruisce un oggetto di tipo `op` (e.g., `Addition`) e si inserisce nello stack

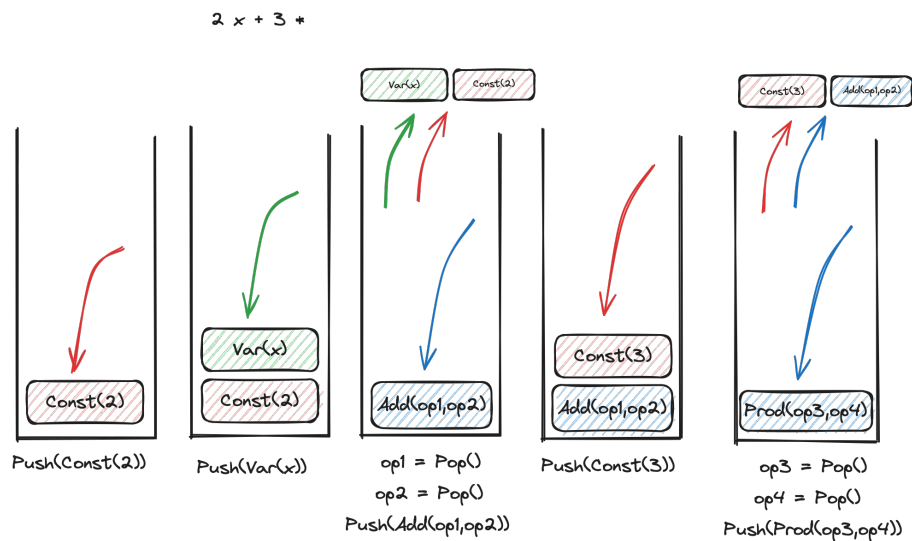


Figure 1: Costruzione di una espressione a partire dalla stringa $2 \times 3 *$

Se la stringa rappresentante l'espressione è corretta allora alla fine della lettura della stringa di input lo stack conterrà un solo elemento corrispondente all'espressione rappresentata dalla stringa.

Implementazione

È necessario implementare le seguenti classi:

- **Expression**. Rappresenta una espressione generica. Deve avere un metodo di classe `from_program(cls, text, dispatch)` che, dato un testo rappresentante espressione in notazione polacca inversa ritorna un oggetto di tipo (una sottoclasse di) espressione.
- **Variable**. Rappresenta una variabile. Il nome è indicato quando viene costruito l'oggetto. Se durante la valutazione l'ambiente non ha un valore per la variabile allora deve essere sollevata l'eccezione `MissingVariableException`.
- **Constant**. Rappresenta una costante il cui valore è fornito al momento della costruzione.
- **Operation**. Rappresenta una operazione astratta. Il costruttore riceve gli argomenti dell'espressione sotto forma di una lista.
- **BinaryOp** e **UnaryOp** specializzano **Operation** rispettivamente ai casi binario e unario
- Le seguenti operazioni binarie (e la corrispondente forma testuale):

Addition	+	$x + t$
Subtraction	-	$x - y$
Multiplication	*	$x \times y$
Division	/	x / y
Modulus	%	$x \bmod y$
Power	**	x^y

- Le seguenti operazioni unarie (e corrispondente forma testuale):

Reciprocal	1/	$1/x$
AbsoluteValue	abs	$ x $

I metodi da implementare sono i seguenti:

- `evaluate(self, env)`: valuta l'espressione usando gli assegnamenti di variabili nel dizionario `env`, che ha associazioni tra nomi di variabili (stringhe) e valori;
- `__str__(self)` per la rappresentazione sotto forma di stringa;
- `op(self, x, y)` (per operazioni binarie) e `op(self, x)` per operazioni unarie. Questi metodi implementano le operazioni definite dalla loro classe (e.g., la somma per **Addition**). Questi metodi hanno senso di esistere solo per le classi che rappresentano operazioni binarie;
- Ogni classe (non singola istanza) specializzata deve avere un variabile `arity` per specificare l'arietà della funzione.

La scelta di dove implementare i singoli metodi è lasciata libera.

Note

- Nella conversione da stringa a espressione l'argomento `dispatch` è un dizionario che associa a delle stringhe la corrispondente classe da utilizzare. Per esempio a `+` assocerà `Addition`. In questo modo diventa possibile costruire facilmente l'espressione corrispondente: quando si incontra `+` si vede che la classe corrispondente ha attributo `arity` pari a 2, quindi si effettua la rimozione di 2 elementi dallo stack, si crea l'oggetto rappresentante l'addizione passando questi due argomenti al costruttore
- Il metodo `split()` disponibile per le stringhe ritorna un array di stringhe ottenute spezzando la stringa di partenza sui separatori (di default lo spazio). Quindi `"a b c".split()` ritorna `['a', 'b', 'c']`. Questo può essere utile a gestire la stringa di input che deve essere trasformata in una espressione.
- Notate che è possibile forzare una sottoclasse (se deve funzionare correttamente) a implementare uno dei metodi della classe base facendo sollevare alla chiamata del metodo l'eccezione `NotImplementedError`.